Solucionario Java — Actividad 1: Resolución de Ejercicios

Curso: Programación Orientada a Objetos II — Semana 02

Este documento desarrolla en Java los dos ejercicios de "Actividad-1-POOII-Resolución de Ejercicios": un sistema de matrícula con herencia (Persona / Alumno / Docente / Curso) y un registro de productos de almacén con interfaz gráfica y persistencia en archivo de texto.

Índice

Ejercicio 1 — Herencia Persona / Alumno / Docente / Curso

- 1.1 Clase Persona.java
- 1.2 Clase Curso.java
- 1.3 Clase Alumno.java
- 1.4 Clase Docente.java
- 1.5 Clase Main.java

Ejercicio 2 — Registro de Productos (GUI + archivo)

- 2.1 Clase Producto.java
- 2.2 Clase RegistroProductoGUI.java

Ejercicio 1 — Herencia Persona / Alumno / Docente / Curso

`Persona` es la superclase de `Alumno` y `Docente` (herencia). `Curso` modela cada curso disponible, que se lee desde un archivo de texto donde cada línea es un curso (cantidad desconocida). Los "cursos disponibles" del archivo son, por diseño del enunciado, los cursos en los que está matriculado el alumno. El constructor de Alumno respeta exactamente el código provisto en la guía de la actividad.

1.1 Clase Persona.java

Atributos comunes `dni`, `nombre` y `apellido`. Incluye un constructor vacío (usado implícitamente por Alumno, cuyo constructor dado no invoca a super(...)) y un constructor completo (usado por Docente).

`src/main/java/poo2/s02/actividad1/Persona.java`

```
package poo2.s02.actividad1;
```

```
public class Persona {

    private String dni;
    private String nombre;
    private String apellido;

    public Persona() {
    }

    public Persona(String dni, String nombre, String apellido) {
        this.dni = dni;
        this.nombre = nombre;
        this.apellido = apellido;
    }

    public String getDni() {
        return dni;
    }

    public void setDni(String dni) {
        this.dni = dni;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
}
```

1.2 Clase Curso.java

Cada línea del archivo de texto se convierte en un objeto Curso cuyo único dato es el nombre.

`src/main/java/poo2/s02/actividad1/Curso.java`

```
package poo2.s02.actividad1;

public class Curso {

    private String nombreCurso;

    public Curso(String nombreCurso) {
        this.nombreCurso = nombreCurso;
    }
```

```
    public String getNombreCurso() {
        return nombreCurso;
    }

    public void setNombreCurso(String nombreCurso) {
        this.nombreCurso = nombreCurso;
    }

    @Override
    public String toString() {
        return nombreCurso;
    }
}
```

1.3 Clase Alumno.java

El constructor es exactamente el que entrega la guía de la actividad (recibe codigo, nombre y listaCursos, y no invoca a super(...), por lo que usa el constructor vacío de Persona). leeCursosAlumno(String archivo) implementa el punto b del enunciado: lee "cursosAlumnos.txt" línea por línea y agrega un Curso por cada línea a listaCursos. mostrarCursos() imprime los cursos en los que está matriculado el alumno, tal como pide el punto 1 de la actividad.

src/main/java/poo2/s02/actividad1/Alumno.java

```
package poo2.s02.actividad1;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class Alumno extends Persona {

    private String codigoAlumno;
    private String nombreAlumno;
    private List<Curso> listaCursos;

    public Alumno(String codigo, String nombre, List<Curso> listaCursos){
        this.codigoAlumno = codigo;
        this.nombreAlumno = nombre;
        this.listaCursos = listaCursos;
        //lee desde un archivo los cursos que corresponden al código del alumno
        //que se recibe como parámetro en el constructor

        this.leeCursosAlumno("cursosAlumnos.txt");
    }

    private void leeCursosAlumno(String archivo) {
        if (listaCursos == null) {
            listaCursos = new ArrayList<>();
        }
        try (BufferedReader br = new BufferedReader(new FileReader(archivo))) {
            String linea;
            while ((linea = br.readLine()) != null) {
```

```
                if (!linea.isBlank()) {
                    listaCursos.add(new Curso(linea.trim()));
                }
            }
        } catch (IOException e) {
            System.out.println("No se pudo leer el archivo de cursos: " +
e.getMessage());
        }
    }

    public void mostrarCursos() {
        System.out.println("Cursos matriculados de " + nombreAlumno + " (" +
codigoAlumno + "):");
        for (Curso curso : listaCursos) {
            System.out.println(" - " + curso);
        }
    }

    public String getCodigoAlumno() {
        return codigoAlumno;
    }

    public void setCodigoAlumno(String codigoAlumno) {
        this.codigoAlumno = codigoAlumno;
    }

    public String getNombreAlumno() {
        return nombreAlumno;
    }

    public void setNombreAlumno(String nombreAlumno) {
        this.nombreAlumno = nombreAlumno;
    }

    public List<Curso> getListaCursos() {
        return listaCursos;
    }

    public void setListaCursos(List<Curso> listaCursos) {
        this.listaCursos = listaCursos;
    }
}
```

1.4 Clase Docente.java

Hereda dni/nombre/apellido de Persona y agrega cursoDocente y facultadDocente. `calificarCurso(Curso)` recibe un Curso y le asigna una nota aleatoria entre 0 y 20 (Random.nextInt(21)), tal como pide el punto g de la actividad.

`src/main/java/poo2/s02/actividad1/Docente.java`

```
package poo2.s02.actividad1;

import java.util.Random;

public class Docente extends Persona {
```

```
    private String cursoDocente;
    private String facultadDocente;

    public Docente(String dni, String nombre, String apellido, String cursoDocente,
String facultadDocente) {
        super(dni, nombre, apellido);
        this.cursoDocente = cursoDocente;
        this.facultadDocente = facultadDocente;
    }

    public int calificarCurso(Curso curso) {
        Random random = new Random();
        int nota = random.nextInt(21); // entero entre 0 y 20
        System.out.println("El docente " + getNombre() + " calificó el curso " +
curso + " con nota: " + nota);
        return nota;
    }

    public String getCursoDocente() {
        return cursoDocente;
    }

    public void setCursoDocente(String cursoDocente) {
        this.cursoDocente = cursoDocente;
    }

    public String getFacultadDocente() {
        return facultadDocente;
    }

    public void setFacultadDocente(String facultadDocente) {
        this.facultadDocente = facultadDocente;
    }
}
```

1.5 Clase Main.java

Crea un Alumno (que internamente carga sus cursos desde cursosAlumnos.txt), muestra sus cursos matriculados y hace que un Docente califique cada uno de ellos.

`src/main/java/poo2/s02/actividad1/Main.java`

```
package poo2.s02.actividad1;

import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {
        List<Curso> cursos = new ArrayList<>();
        Alumno alumno = new Alumno("A-001", "Sofia Ramos", cursos);
        alumno.mostrarCursos();

        System.out.println();

        Docente docente = new Docente("12345678", "Carlos", "Vega", "Programación
```

```
II", "Ingeniería");
        for (Curso curso : alumno.getListaCursos()) {
            docente.calificarCurso(curso);
        }
    }
}
```

Archivo de datos `cursosAlumnos.txt` de ejemplo (debe ubicarse en el directorio de ejecución del programa), un curso por línea:

`cursosAlumnos.txt`

```
Programación Orientada a Objetos II
Base de Datos I
Arquitectura de Software
```

Volver al índice · Siguiente: Ejercicio 2 — Registro de Productos (GUI + archivo)

Ejercicio 2 — Registro de Productos (GUI + archivo)

Se implementa una interfaz gráfica en Swing con campos para nombre y cantidad. Al registrar un producto se agrega a una lista de objetos Producto y, en cada registro, se reescribe por completo el archivo "stock.txt" con todos los productos ingresados hasta el momento (punto b de la actividad).

2.1 Clase Producto.java

`src/main/java/poo2/s02/actividad2/Producto.java`

```
package poo2.s02.actividad2;

public class Producto {

    private String nombreProducto;
    private int cantidad;

    public Producto(String nombreProducto, int cantidad) {
        this.nombreProducto = nombreProducto;
        this.cantidad = cantidad;
    }

    public String getNombreProducto() {
        return nombreProducto;
    }

    public void setNombreProducto(String nombreProducto) {
        this.nombreProducto = nombreProducto;
    }
```

```
    public int getCantidad() {
        return cantidad;
    }

    public void setCantidad(int cantidad) {
        this.cantidad = cantidad;
    }

    @Override
    public String toString() {
        return nombreProducto + " - " + cantidad + " unidad(es)";
    }
}
```

2.2 Clase RegistroProductoGUI.java

Formulario con dos campos (nombre y cantidad) y un botón "Registrar producto". Cada registro válido: crea el Producto, lo agrega a la lista, lo muestra en pantalla y guarda toda la lista en stock.txt (una línea por producto, formato nombre;cantidad).

src/main/java/poo2/s02/actividad2/RegistroProductoGUI.java

```
package poo2.s02.actividad2;

import javax.swing.*;
import java.awt.*;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
import java.util.List;

public class RegistroProductoGUI extends JFrame {

    private final List<Producto> productos = new ArrayList<>();

    private final JTextField txtNombre = new JTextField(15);
    private final JTextField txtCantidad = new JTextField(5);
    private final DefaultListModel<String> modeloLista = new DefaultListModel<>();

    public RegistroProductoGUI() {
        super("Registro de Productos - Almacén");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout(10, 10));

        JPanel panelFormulario = new JPanel(new GridLayout(2, 2, 5, 5));
        panelFormulario.add(new JLabel("Nombre del producto:"));
        panelFormulario.add(txtNombre);
        panelFormulario.add(new JLabel("Cantidad:"));
        panelFormulario.add(txtCantidad);

        JButton btnRegistrar = new JButton("Registrar producto");
        btnRegistrar.addActionListener(e -> registrarProducto());

        JList<String> listaProductos = new JList<>(modeloLista);

        add(panelFormulario, BorderLayout.NORTH);
```

```
        add(new JScrollPane(listaProductos), BorderLayout.CENTER);
        add(btnRegistrar, BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null);
    }

    private void registrarProducto() {
        String nombre = txtNombre.getText().trim();
        String textoCantidad = txtCantidad.getText().trim();
        if (nombre.isEmpty() || textoCantidad.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Completa nombre y cantidad.");
            return;
        }
        try {
            int cantidad = Integer.parseInt(textoCantidad);
            Producto producto = new Producto(nombre, cantidad);
            productos.add(producto);
            modeloLista.addElement(producto.toString());
            guardarEnArchivo();
            txtNombre.setText("");
            txtCantidad.setText("");
        } catch (NumberFormatException ex) {
            JOptionPane.showMessageDialog(this, "La cantidad debe ser un número
entero.");
        }
    }

    private void guardarEnArchivo() {
        try (PrintWriter writer = new PrintWriter(new FileWriter("stock.txt"))) {
            for (Producto producto : productos) {
                writer.println(producto.getNombreProducto() + ";" +
producto.getCantidad());
            }
        } catch (IOException e) {
            JOptionPane.showMessageDialog(this, "No se pudo guardar el archivo
stock.txt: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
RegistroProductoGUI().setVisible(true));
    }
}
```

Volver al índice